

# Thực hành – Tuần 2B

## Tên & Môi trường



Viện Trí tuệ Nhân tạo, Trường Đại học Công nghệ, ĐHQGHN

Ngày 16 tháng 3 năm 2026

## Mục lục

|            |                                                                  |           |
|------------|------------------------------------------------------------------|-----------|
| <b>I</b>   | <b>Nội dung thực hành</b>                                        | <b>2</b>  |
| I.1        | Kiến thức . . . . .                                              | 2         |
| I.1.1      | Tên (Name) và Đối tượng (Object) trong C++ . . . . .             | 2         |
| I.1.2      | Thời điểm liên kết (Binding Time) trong C++ . . . . .            | 2         |
| I.1.3      | Định nghĩa rõ (Explicit) và Định nghĩa ngầm (Implicit) . . . . . | 3         |
| I.1.4      | Phạm vi (Scope) trong C++ . . . . .                              | 4         |
| I.1.5      | Khối lệnh (Block) trong C++ . . . . .                            | 5         |
| I.1.6      | Vòng đời của biến (Variable Lifetime) . . . . .                  | 5         |
| I.1.7      | Bí danh (Aliasing) trong C++ . . . . .                           | 6         |
| I.2        | Kiểu dữ liệu cơ bản trong C++ . . . . .                          | 7         |
| I.3        | Biến (Variables) . . . . .                                       | 9         |
| I.4        | Hằng số (Constants) . . . . .                                    | 10        |
| I.5        | Nhập xuất cơ bản . . . . .                                       | 10        |
| I.6        | Hàm (Functions) . . . . .                                        | 11        |
| I.7        | Mục tiêu . . . . .                                               | 13        |
| <b>II</b>  | <b>Bài tập thực hành</b>                                         | <b>14</b> |
| <b>III</b> | <b>Bài tập về nhà</b>                                            | <b>18</b> |

# I. Nội dung thực hành

## I.1 Kiến thức

Trong phần lý thuyết trước đó, các khái niệm như *tên (name)*, *đối tượng (object)*, *liên kết (binding)*, *môi trường (environment)* và *phạm vi (scope)* đã được giới thiệu ở mức trừu tượng. Trong phần này, chúng ta sẽ ánh xạ các khái niệm đó sang ngôn ngữ C++ để hiểu cách chúng xuất hiện trong chương trình thực tế.

### I.1.1 Tên (Name) và Đối tượng (Object) trong C++

**Tên (name)** là định danh được sử dụng trong chương trình để biểu diễn một đối tượng. **Đối tượng (object)** là một vùng bộ nhớ chứa dữ liệu.

Ví dụ trong C++:

```
1 int x = 10;
```

Trong câu lệnh trên:

- x là **name**
- giá trị 10 được lưu trong một **object** (vùng nhớ)
- quá trình liên kết x với vùng nhớ đó được gọi là **binding**

Ví dụ khác:

```
1 int x = 5;  
2 int y = x;
```

Ở đây:

- x và y là hai **name**
- mỗi biến tương ứng với một **object** trong bộ nhớ

### I.1.2 Thời điểm liên kết (Binding Time) trong C++

**Liên kết (binding)** là sự liên kết giữa một *tên (name)* và một *đối tượng (object)*. Trong quá trình thực thi chương trình, việc liên kết có thể xảy ra ở các thời điểm khác nhau.

| Thời điểm            | Ví dụ trong C++             |
|----------------------|-----------------------------|
| Language design time | các kiểu như int, double    |
| Compile time         | biến toàn cục, kiểu dữ liệu |
| Runtime              | biến cục bộ, cấp phát động  |

Ví dụ:

```
1 int global = 10;
2
3 int main() {
4     int local = 5;
5 }
```

Trong chương trình trên:

- `global` được liên kết khi chương trình được biên dịch
- `local` được tạo ra khi hàm `main()` được thực thi

### 1.1.3 Định nghĩa rõ (Explicit) và Định nghĩa ngầm (Implicit)

Trong nhiều ngôn ngữ lập trình, việc tạo liên kết giữa một *tên (name)* và một *đối tượng (object)* thường xảy ra thông qua một **khai báo (declaration)**.

Có hai cách phổ biến:

- **Định nghĩa rõ (explicit declaration)**: tên, kiểu dữ liệu và thông tin liên quan phải được khai báo trước khi sử dụng.
- **Định nghĩa ngầm (implicit declaration)**: liên kết giữa tên và đối tượng được tạo ra tự động khi tên đó được sử dụng lần đầu.

**Ví dụ với Python (Implicit)** Python cho phép tạo biến mà không cần khai báo kiểu trước.

```
1 x = 10
2 y = 3.14
3 name = "Alice"
```

Ở đây:

- biến được tạo ra ngay khi xuất hiện trong chương trình
- kiểu dữ liệu được suy ra tự động từ giá trị
- lập trình viên không cần khai báo trước

Cơ chế này giúp chương trình ngắn gọn hơn nhưng cũng có thể dẫn đến lỗi phát hiện muộn ở thời điểm thực thi.

**Ví dụ với C++ (Explicit)** Ngược lại, C++ yêu cầu phải khai báo kiểu dữ liệu trước khi sử dụng biến.

```
1 int x = 10;
2 double y = 3.14;
3 std::string name = "Alice";
```

Trong trường hợp này:

- mỗi biến phải có kiểu dữ liệu rõ ràng
- trình biên dịch kiểm tra kiểu ngay trong quá trình biên dịch

Việc khai báo tường minh giúp chương trình:

- phát hiện lỗi sớm hơn
- hỗ trợ tối ưu hóa tốt hơn
- làm cho cấu trúc chương trình rõ ràng hơn

Một dạng trung gian trong C++ là từ khóa `auto`, cho phép trình biên dịch suy diễn kiểu dữ liệu:

```
1 auto x = 10;  
2 auto y = 3.14;
```

Tuy nhiên, việc suy diễn kiểu này vẫn được thực hiện tại **compile time**, nên vẫn thuộc nhóm định nghĩa rõ trong ngôn ngữ tĩnh kiểu như C++.

#### 1.1.4 Phạm vi (Scope) trong C++

**Phạm vi (scope)** xác định vùng của chương trình mà một tên có thể được sử dụng.

C++ sử dụng **phạm vi tĩnh (static scope)** hay còn gọi là **phạm vi từ vựng (lexical scope)**, nghĩa là phạm vi được xác định dựa trên cấu trúc văn bản của chương trình.

Ví dụ:

```
1 #include <iostream>  
2  
3 int x = 10;  
4  
5 int main() {  
6     int x = 20;  
7     std::cout << x << std::endl;  
8 }
```

Kết quả:

```
1 20
```

Ở đây:

- biến `x` toàn cục có giá trị 10
- biến `x` trong hàm `main` có giá trị 20
- biến bên trong đã **che khuất (shadowing)** biến bên ngoài

### 1.1.5 Khối lệnh (Block) trong C++

Một **khối lệnh (block)** là một vùng mã được giới hạn bởi dấu { và }.

Block thường được dùng để:

- nhóm các câu lệnh
- tạo phạm vi cho biến cục bộ

Ví dụ:

```
1 #include <iostream>
2
3 int main() {
4
5     int x = 10;
6
7     {
8         int y = 5;
9         std::cout << x << std::endl;
10    }
11
12 }
```

Trong ví dụ này:

- x có phạm vi trong toàn bộ hàm main
- y chỉ tồn tại bên trong block bên trong

### 1.1.6 Vòng đời của biến (Variable Lifetime)

**Vòng đời (lifetime)** của một biến là khoảng thời gian mà *đối tượng (object)* tương ứng tồn tại trong bộ nhớ trong quá trình thực thi chương trình. Trong C++, vòng đời của biến phụ thuộc vào nơi biến được khai báo.

**Biến toàn cục (global variable)** Biến toàn cục được khai báo bên ngoài mọi hàm. Những biến này được tạo ra khi chương trình bắt đầu và tồn tại cho đến khi chương trình kết thúc.

```
1 #include <iostream>
2
3 int global = 10;
4
5 int main() {
6     std::cout << global << std::endl;
7 }
```

Trong ví dụ trên, biến `global` tồn tại trong suốt thời gian thực thi của chương trình.

**Biến địa phương (local variable)** Biến địa phương được khai báo bên trong một hàm hoặc một block. Những biến này được tạo ra khi khối lệnh bắt đầu thực thi và bị hủy khi khối lệnh kết thúc.

```
1 void foo() {  
2     int x = 5;  
3 }
```

Biến x được tạo khi hàm foo() bắt đầu thực thi và bị hủy khi hàm kết thúc.

**Tham số hàm (function parameter)** Tham số của hàm cũng là một dạng biến địa phương. Chúng được tạo ra khi hàm được gọi và bị hủy khi hàm kết thúc.

```
1 void printValue(int value) {  
2     std::cout << value << std::endl;  
3 }
```

Trong trường hợp này, biến value chỉ tồn tại trong thời gian hàm printValue() đang được thực thi.

**Vòng đời của liên kết (Binding) và vòng đời của đối tượng (Object)** Trong nhiều trường hợp, vòng đời của *liên kết (binding)* giữa tên và đối tượng và vòng đời của *đối tượng (object)* là giống nhau. Ví dụ, khi khai báo một biến địa phương, việc tạo biến và tạo liên kết với tên thường diễn ra cùng lúc.

Tuy nhiên, hai vòng đời này không phải lúc nào cũng hoàn toàn trùng nhau. Trong một số tình huống như truyền tham chiếu hoặc sử dụng con trỏ, một đối tượng có thể tồn tại lâu hơn liên kết tạm thời tới nó, hoặc ngược lại liên kết vẫn tồn tại nhưng đối tượng đã bị hủy. Những trường hợp này có thể dẫn đến các lỗi như *dangling reference*.

### 1.1.7 Bí danh (Aliasing) trong C++

**Bí danh (aliasing)** xảy ra khi nhiều tên cùng tham chiếu tới một đối tượng trong bộ nhớ.

Ví dụ sử dụng con trỏ:

```
1 int x = 10;  
2  
3 int* p = &x;  
4 int* q = &x;
```

Trong trường hợp này:

- p và q đều trỏ tới cùng một biến x
- do đó hai tên này là **aliases** của cùng một đối tượng

Việc aliasing cần được sử dụng cẩn thận vì thay đổi thông qua một tên có thể ảnh hưởng tới tất cả các tên khác cùng trỏ tới đối tượng đó.

## 1.2 Kiểu dữ liệu cơ bản trong C++

Trong C++, mỗi biến phải được khai báo với một **kiểu dữ liệu** (data type). Kiểu dữ liệu xác định:

- kích thước bộ nhớ được sử dụng
- miền giá trị có thể biểu diễn
- các phép toán hợp lệ có thể áp dụng

Khác với Python, C++ là ngôn ngữ **static typing**, nghĩa là kiểu của biến phải được xác định tại thời điểm biên dịch.

### Kiểu số nguyên (Integer Types)

Các kiểu số nguyên dùng để lưu trữ số nguyên.

| Kiểu      | Kích thước điển hình   | Miền giá trị                       |
|-----------|------------------------|------------------------------------|
| short     | 2 bytes                | $\approx -32\,768$ đến $32\,767$   |
| int       | 4 bytes                | $\approx -2^{31}$ đến $2^{31} - 1$ |
| long      | 8 bytes (tùy hệ thống) | rất lớn                            |
| long long | 8 bytes                | $\approx -2^{63}$ đến $2^{63} - 1$ |

Ví dụ:

```
1 int a = 10;  
2 long b = 1000000;  
3 long long c = 90000000000;
```

Có thể khai báo số nguyên không dấu bằng từ khóa unsigned:

```
1 unsigned int count = 100;
```

### Kiểu số thực (Floating Point)

Các kiểu này dùng để biểu diễn số thực.

| Kiểu        | Độ chính xác           |
|-------------|------------------------|
| float       | khoảng 7 chữ số        |
| double      | khoảng 15 chữ số       |
| long double | cao hơn (tùy hệ thống) |

Ví dụ:

```
1 float x = 3.14f;
2 double y = 2.71828;
```

Trong thực tế, `double` thường được sử dụng phổ biến hơn `float`.

## Kiểu ký tự (Character)

Kiểu `char` dùng để lưu trữ một ký tự đơn.

```
1 char c = 'A';
2 char digit = '5';
```

Giá trị thực tế được lưu dưới dạng mã ASCII.

## Kiểu Boolean

Kiểu `bool` biểu diễn giá trị logic.

```
1 bool flag = true;
2 bool done = false;
```

Trong nhiều phép toán, giá trị:

- `true` tương đương 1
- `false` tương đương 0

## Chuỗi ký tự

Trong C++, chuỗi thường được sử dụng thông qua thư viện chuẩn `std::string`.

```
1 #include <string>
2
3 std::string name = "Alice";
```

Điều này khác với Python, nơi chuỗi là kiểu dữ liệu nguyên thủy của ngôn ngữ.

## Suy diễn kiểu với *auto*

C++ cho phép trình biên dịch suy diễn kiểu dữ liệu thông qua từ khóa `auto`.

```
1 auto x = 10;           // int
2 auto y = 3.14;         // double
3 auto name = "Bob";
```

Kiểu dữ liệu vẫn được xác định tại **compile time**.



## Toán tử số học cơ bản

Các toán tử phổ biến trong C++ tương tự nhiều ngôn ngữ khác như Python.

| Toán tử | Ý nghĩa     |
|---------|-------------|
| +       | cộng        |
| -       | trừ         |
| *       | nhân        |
| /       | chia        |
| %       | chia lấy dư |

Ví dụ:

```
1 int a = 10;  
2 int b = 3;  
3  
4 int sum = a + b;  
5 int mod = a % b;
```

## So sánh nhanh với Python

| Đặc điểm       | Python     | C++                    |
|----------------|------------|------------------------|
| Khai báo kiểu  | Không cần  | Bắt buộc               |
| Kiểu số nguyên | Tự mở rộng | Có giới hạn kích thước |
| Chuỗi          | built-in   | std::string            |
| Suy diễn kiểu  | Runtime    | Compile time (auto)    |

Những kiểu dữ liệu trên là các kiểu được sử dụng thường xuyên nhất trong các bài tập lập trình cơ bản với C++.

### 1.3 Biến (Variables)

Trong C++, một biến cần được **khai báo** trước khi sử dụng. Việc khai báo xác định tên biến và kiểu dữ liệu của nó.

```
1 int x;  
2 double y;
```

Sau khi khai báo, biến có thể được **khởi tạo** (gán giá trị ban đầu).

```
1 int x = 10;  
2 double y = 3.14;
```

Trong nhiều trường hợp, khai báo và khởi tạo thường được thực hiện cùng lúc:

```
1 int a = 5;
2 int b = 8;
3 int sum = a + b;
```

C++ cũng hỗ trợ suy diễn kiểu dữ liệu bằng từ khóa `auto`:

```
1 auto x = 10;    // int
2 auto y = 3.14;  // double
```

Trình biên dịch sẽ suy ra kiểu dữ liệu tại **compile time**.

## 1.4 Hằng số (Constants)

Hằng số là những giá trị không thể thay đổi sau khi được khởi tạo. Trong C++, hằng số được khai báo bằng từ khóa `const`.

```
1 const int MAX = 100;
2 const double PI = 3.14159;
```

Sau khi khai báo, giá trị của hằng số không thể thay đổi:

```
1 MAX = 200;    // compile error
```

Việc sử dụng `const` giúp:

- tránh thay đổi giá trị ngoài ý muốn
- làm rõ ý nghĩa của dữ liệu trong chương trình
- hỗ trợ trình biên dịch tối ưu hóa

**So sánh với Python** Trong Python, ngôn ngữ không có từ khóa riêng để khai báo hằng số. Theo quy ước, các hằng số thường được viết bằng chữ in hoa:

```
1 MAX = 100
2 PI = 3.14159
```

Tuy nhiên, Python không ngăn cản việc thay đổi giá trị:

```
1 MAX = 200    # not error
```

Do đó, trong Python việc sử dụng chữ in hoa chỉ là **quy ước lập trình**, trong khi C++ sử dụng `const` để **bắt buộc hằng số không được thay đổi** ngay từ thời điểm biên dịch.

## 1.5 Nhập xuất cơ bản

Trong C++, thư viện chuẩn `iostream` được sử dụng cho các thao tác nhập và xuất dữ liệu.

**Xuất dữ liệu** Sử dụng `std::cout` để in dữ liệu ra màn hình.

```
1 #include <iostream>
2
3 int main() {
4     int x = 10;
5     std::cout << x << std::endl;
6 }
```

Toán tử `<<` được dùng để đưa dữ liệu ra luồng xuất.

**Nhập dữ liệu** Sử dụng `std::cin` để đọc dữ liệu từ bàn phím.

```
1 #include <iostream>
2
3 int main() {
4     int x;
5     std::cin >> x;
6 }
```

Toán tử `>>` được dùng để đọc dữ liệu từ luồng nhập.

**Ví dụ** Ta có chương trình nhập hai số nguyên a và b. In ra tổng hai số như sau:

```
1 #include <iostream>
2
3 int main() {
4     int a, b;
5
6     std::cin >> a >> b;
7
8     std::cout << "Sum = " << a + b << std::endl;
9 }
```

## 1.6 Hàm (Functions)

Trong C++, chương trình thường được tổ chức thành các **hàm** (function). Mỗi hàm thực hiện một nhiệm vụ cụ thể và có thể được gọi từ những phần khác của chương trình.

Một hàm bao gồm:

- kiểu dữ liệu trả về (return type)
- tên hàm
- danh sách tham số (parameters)
- thân hàm (function body)

## Khai báo và định nghĩa hàm

Ví dụ một hàm tính tổng hai số nguyên:

```
1 int add(int a, int b) {  
2     return a + b;  
3 }
```

Trong đó:

- int đầu tiên là kiểu trả về
- add là tên hàm
- a, b là các tham số

Hàm có thể được gọi trong chương trình:

```
1 int result = add(3, 5);
```

## Tham số hàm

Tham số của hàm là các biến được tạo ra khi hàm được gọi.

```
1 int square(int x) {  
2     return x * x;  
3 }
```

Khi gọi:

```
1 int y = square(4);
```

Biến x chỉ tồn tại trong thời gian hàm square thực thi.

## Truyền tham trị (Pass by Value)

Mặc định trong C++, tham số được truyền theo **tham trị**.

Điều này nghĩa là hàm nhận một **bản sao** của giá trị.

```
1 void increase(int x) {  
2     x = x + 1;  
3 }  
4  
5 int main() {  
6     int a = 10;  
7     increase(a);  
8 }
```

Sau khi gọi hàm, giá trị của a vẫn là 10.

## Truyền tham chiếu (Pass by Reference)

C++ cho phép truyền tham chiếu bằng ký hiệu &.

```
1 void increase(int& x) {  
2     x = x + 1;  
3 }  
4  
5 int main() {  
6     int a = 10;  
7     increase(a);  
8 }
```

Trong trường hợp này, x là **tham chiếu** tới biến a, nên giá trị của a sẽ thay đổi thành 11.

## So sánh với Python

Trong Python, các tham số được truyền theo cơ chế **object reference**.

Ví dụ:

```
1 def increase(x):  
2     x = x + 1  
3  
4 a = 10  
5 increase(a)
```

Giá trị của a vẫn là 10. Điều này tương tự với cơ chế **pass by value** khi làm việc với các kiểu dữ liệu bất biến (immutable) như số nguyên.

## I.7 Mục tiêu

Trong bài tập tuần này, các bạn sẽ thực hành việc đặt tên, tạo liên kết và tương tác với môi trường thông qua các biến trong chương trình C++. Bằng cách áp dụng kiến thức về kiểu dữ liệu, phạm vi và vòng đời của biến, các bạn sẽ thực hành khai báo, khởi tạo và sử dụng biến trong các chương trình đơn giản với luồng nhập xuất cơ bản.

Cụ thể, sau khi hoàn thành bài tập, sinh viên cần đạt được các mục tiêu sau:

- Hiểu và áp dụng các khái niệm cơ bản như *tên (name)*, *đối tượng (object)*, *liên kết (binding)*, *phạm vi (scope)* và *vòng đời (lifetime)* của biến trong chương trình C++.
- Sử dụng được các **kiểu dữ liệu cơ bản** trong C++ như `int`, `double`, `char`, `bool` và `std::string`.
- Thực hành **khai báo và khởi tạo biến**, sử dụng **hằng số** với từ khóa `const`, và hiểu vai trò của kiểu dữ liệu tĩnh trong C++.

- Sử dụng các thao tác **nhập xuất cơ bản** thông qua `std::cin` và `std::cout`.
- Phân tích được phạm vi và vòng đời của biến khi chương trình sử dụng nhiều khối lệnh và nhiều hàm khác nhau.
- Hiểu cách **tham số hàm** hoạt động như một dạng biến địa phương và cách dữ liệu được truyền giữa các hàm.

## II. Bài tập thực hành

### Bài tập 1: Biến và Kiểu dữ liệu cơ bản

#### Mô tả:

Viết một chương trình C++ khai báo các biến với các kiểu dữ liệu cơ bản sau:

- `int age`
- `double height`
- `char grade`
- `bool isStudent`

Gán giá trị phù hợp cho các biến và in các giá trị này ra màn hình.

#### Yêu cầu kỹ thuật:

- Sử dụng `std::cout` để in kết quả
- Mỗi thông tin in trên một dòng

#### Input:

Không có dữ liệu đầu vào.

#### Output:

Kết quả gồm 4 dòng như sau:

```
Age: 20
Height: 1.75
Grade: A
Student: true
```

## Bài tập 2: Nhập xuất cơ bản và Toán tử số học

### Mô tả:

Viết chương trình nhập hai số nguyên a và b từ bàn phím. Sau đó in ra các kết quả sau:

- tổng của hai số
- hiệu của hai số
- tích của hai số
- thương của hai số (chia nguyên)
- phần dư của phép chia

### Yêu cầu kỹ thuật:

- Sử dụng `std::cin` để nhập dữ liệu
- Sử dụng các toán tử số học của C++

### Input:

Hai số nguyên cách nhau bởi dấu cách:

```
10 3
```

### Output:

Kết quả gồm 5 dòng như sau:

```
Sum = 13
Diff = 7
Product = 30
Quotient = 3
Remainder = 1
```

## Bài tập 3: Sử dụng Hằng số

### Mô tả:

Viết chương trình tính chu vi và diện tích của hình tròn. Chương trình thực hiện:

- Nhập bán kính r từ bàn phím
- Sử dụng hằng số PI để tính toán

### Yêu cầu kỹ thuật:

- Khai báo PI bằng từ khóa const
- Sử dụng công thức:
  - Chu vi:  $2 \times PI \times r$
  - Diện tích:  $PI \times r^2$

### Gợi ý:

```
1 const double PI = 3.14159;
```

### Input:

Một số thực là bán kính:

5

### Output:

Kết quả gồm 2 dòng như sau:

```
Circumference = 31.4159
Area = 78.5398
```

## Bài tập 4: Phạm vi (Scope) và Shadowing

### Mô tả:

Cho chương trình sau:

```
1 #include <iostream>
2
3 int x = 10;
4
5 int main() {
6
7     std::cout << x << std::endl;
8
9     int x = 20;
10    std::cout << x << std::endl;
11
12    {
13        int x = 30;
14        std::cout << x << std::endl;
```



```
15     }  
16  
17 }
```

Chương trình trên sử dụng ba phạm vi (scope) khác nhau của biến x:

- biến toàn cục (global variable)
- biến địa phương trong hàm main
- biến trong một khối lệnh (block)

#### **Yêu cầu:**

- Dự đoán kết quả chương trình trước khi chạy
- Sau đó biên dịch và chạy chương trình để kiểm tra kết quả

#### **Câu hỏi thảo luận:**

1. Vì sao cùng một tên biến x nhưng mỗi lần `std::cout` lại in ra giá trị khác nhau?
2. Điều gì xảy ra nếu ta thêm dòng sau vào cuối hàm main?

```
1 std::cout << x << std::endl;
```

Giá trị nào sẽ được in ra? Vì sao?

### **Bài tập 5: Phạm vi (Scope) và Vòng đời (Lifetime)**

#### **Mô tả:**

Cho chương trình sau:

```
1 #include <iostream>  
2  
3 void write(int x) {  
4     std::cout << x << std::endl;  
5 }  
6  
7 void fie() {  
8     write(n);  
9 }  
10  
11 void foo() {  
12     int n = 5;
```

```

13     write(n);
14 }
15
16 int main() {
17
18     int n = 10;
19
20     fie();
21     foo();
22
23 }

```

#### Yêu cầu:

1. Hãy xác định xem chương trình trên có biên dịch được hay không. Giải thích lý do.
2. Nếu chương trình không biên dịch được, hãy sửa chương trình để hàm `fie()` có thể in ra giá trị của biến `n` trong `main`.
3. Sau khi sửa, dự đoán kết quả chương trình khi chạy.

#### Câu hỏi thảo luận:

1. Vì sao biến `n` trong `main()` không thể được sử dụng trực tiếp trong hàm `fie()`?
2. Tham số `x` của hàm `write` có **scope** và **lifetime** như thế nào?
3. Biến `n` trong `foo()` và biến `n` trong `main()` có liên quan gì tới nhau không?

## III. Bài tập về nhà

Làm các bài tập chủ đề **Tên & Môi trường** (*[Name & Environments]*) trên trang Coding. Liên hệ với giảng viên thực hành về thời hạn làm bài.

**Nhắc lại:** Để biên dịch và thực thi bài làm bằng C++, trong mã nguồn làm bài sẽ cần để ở dòng đầu tiên đoạn mã `// .cpp`, ví dụ như:

```

1 // .cpp

```

```
2 #include <iostream>
3 using namespace std;
4 ...
```